

Esame OS Internals (Cabodi)

Data: 18/10/2021

QUESTIONS

SECTION 1: THEORETICAL (Silberschatz)

Domanda 1 (Memoria Virtuale, TLB e EAT)

Contesto: Sia dato un sistema di memoria virtuale con paginazione, nel quale vengono indirizzati i Byte. Il sistema dispone di TLB (Translation Look-aside Buffer), su cui si misura sperimentalmente un "hit ratio" H . La tabella delle pagine ("page-table") viene realizzata con uno schema a due livelli, nel quale un indirizzo logico di 64 bit viene suddiviso (da MSB a LSB) in 3 parti: $p1$, $p2$ e d , rispettivamente di 40 bit, 12 bit e 12 bit. Non si utilizzano ulteriori strutture dati (quali tabelle di hash o inverted page table) per velocizzare gli accessi. La memoria virtuale viene gestita con paginazione a richiesta.

Si risponda alle seguenti domande:

-) Supponendo che la memoria RAM abbia tempo di accesso di $T_{RAM} = 200$ ns, si calcoli H tale da garantire tempo effettivo di accesso (EAT) minore di 250 ns, assumendo che il tempo di accesso alla TLB sia trascurabile.
 -) Si consideri ora la frequenza di page fault $p = 10^{-5}$. In quanto tempo T viene servito un page fault se si osserva che il degrado nel tempo di accesso dovuto ai soli PF è superiore al 10%? Si tratta di un valore massimo o minimo per T ? (Siccome H non è noto, ma è dato da un intervallo di valori, si calcoli T per H minimo e massimo).
 -) Data una stringa di riferimenti a indirizzi in memoria (letture e scritture), il numero di page fault può variare al variare della dimensione di una pagina? (Motivare). A parità di tutto il resto, può servire modificare la dimensione (numero di bit) di $p1$ e $p2$ (senza cambiare d) per ridurre il numero di PF? (Motivare).
-

Domanda 2 (File System Unix-like, Inodes e Blocchi)

Contesto: Considerare un file system Unix-like, basato su inode, con 13 puntatori / indici (10 diretti, 1 singolo indiretto, 1 doppio indiretto e 1 triplo indiretto). I puntatori / indici hanno una dimensione di 32 bit e i blocchi del disco hanno una dimensione di X Byte. Il file system risiede su una partizione del disco di 800 GB, che include sia blocchi di dato che blocchi di indice.

Si risponda alle seguenti domande:

-) Supponendo che tutti i metadati (eccetto i blocchi indice) abbiano dimensioni trascurabili, calcolare i possibili valori di X , tenendo conto che si tratta di una potenza di 2 e che il file system deve garantire di poter contenere fino a 100 file con indicizzazione indiretta tripla. Si calcolino i valori minimo e massimo per X .
-) Dato un file binario di dimensione 10381.25 KB, calcolare esattamente quanti blocchi indice e blocchi di dato occupa il file, corrispondenti ai valori minimo e massimo precedentemente calcolati per X .

Domanda 3 (Link Dinamico, IPT e TLB)

Contesto: Si risponda alle seguenti domande sulla gestione della memoria:

-) Si consideri il caricamento dinamico (dynamic loading) e il link dinamico (dynamic linking). È possibile caricare dinamicamente un programma senza che sia necessario il dynamic linking? Il dynamic linking richiede che un programma sia anche caricabile dinamicamente (dynamic loading)?
-) Si spieghi brevemente perché un’Inverted Page Table (IPT) necessiti di una tabella di HASH. Perché la soluzione di IPT + tabella di HASH è diversa da una soluzione con PT basata solamente su tabella di Hash?
-) Si consideri una CPU dotata di TLB: la TLB può contenere entry di più processi simultaneamente o è vincolata a contenere entry per un solo processo?

SECTION 2: OS161

Domanda 4 (Configurazione e Compilazione Kernel)

Contesto: Sia dato un sistema OS161. Si supponga di aver aggiunto a `kern/conf/conf.kern` le istruzioni:

```
defoption exam
optfile exam syscall/exam.c
```

e di aver creato in `kern/conf` il file `EXAM`, copiato dal file `DUMBVM`.

Si risponda alle seguenti domande:

-) Si dica se sono sufficienti le azioni appena descritte, più l'esecuzione in `kern/conf` di `./config EXAM`, affinché venga compilato il file opzionale `syscall/exam.c` quando si effettuano, in `kern/compile/EXAM`, i comandi `bmake depend` e `bmake`.
-) Quale, tra i file `exam.h` e `opt-exam.h`, viene generato automaticamente dal comando `./config EXAM`? Viene generato sempre, oppure solo se in `EXAM` compare l'istruzione `options exam`?
-) Cosa contiene il file (quello generato automaticamente, di cui alla domanda precedente)?
-) Si supponga di inserire, nel file `main.c`, l'istruzione `exam_init();`. Tenendo conto che è realizzata nel file `syscall/exam.c`, come è possibile fare in modo che l'istruzione sia presa in considerazione e compilata solamente nelle versioni del kernel in cui viene abilitata l'opzione `exam`, mediante le istruzioni seguenti?

```
#ifdef OPT_EXAM
    exam_init();
#endif
```

Dire anche cosa è necessario includere in un file C per poter usare questo tipo di opzioni.

Domanda 5 (Processi, Trapframe e I/O)

Contesto: Si consideri un processo utente su OS161.

Si risponda alle seguenti domande:

-) Spiegare brevemente la differenza tra `switchframe` e `trapframe`. Quale viene utilizzato per avviare un processo utente? Quale per gestire una chiamata di sistema? Quale per la `thread_fork`?
-) Perché i dispositivi di I/O sono mappati su `kseg1`? Sarebbe possibile mappare un dispositivo di I/O in `kseg0`?
-) Un processo utente può eseguire una `write(fd, buf, nb)`, dove `buf` è un indirizzo in `kseg0`?

ANSWERS

Domanda 1 (Memoria Virtuale, TLB e EAT)

A) Calcolo di H per garantire $EAT < 250$ ns:

In un sistema con Page Table a due livelli ($N = 2$), un TLB Miss richiede 2 accessi addizionali in memoria per navigare la struttura ad albero, prima di poter accedere al dato. Assumendo il tempo di accesso alla TLB nullo:

- Tempo TLB Hit = $T_{RAM} = 200$ ns
- Tempo TLB Miss = $2 \cdot T_{RAM} + T_{RAM} = 3 \cdot T_{RAM} = 600$ ns

L'Effective Access Time (EAT) è:

$$EAT = H \cdot 200 + (1 - H) \cdot 600 = 600 - 400H$$

Imponendo il vincolo richiesto:

$$600 - 400H < 250 \implies 400H > 350 \implies H > \frac{350}{400} \implies \mathbf{H > 0.875 \text{ (87,5\%)}}$$

B) Calcolo del tempo di servizio del Page Fault (T):

Dal punto precedente ricaviamo l'intervallo di EAT_{base} a seconda dell'hit ratio:

- $EAT_{min} = 200$ ns (con $H = 1$)
- $EAT_{max} = 250$ ns (con $H = 0.875$)

La formula con degrado è $EAT_{PF} = (1 - p)EAT_{base} + p \cdot T$. Il degrado causato dai PF deve essere $> 10\%$ di EAT_{base} , quindi:

$$EAT_{PF} - EAT_{base} > 0.1 \cdot EAT_{base} \implies p(T - EAT_{base}) > 0.1 \cdot EAT_{base} \implies T > EAT_{base} \cdot \left(\frac{0.1 + p}{p} \right)$$

Sapendo che $p = 10^{-5}$:

1. Caso H minimo ($EAT = 250$ ns):

$$T > 250 \cdot \left(\frac{0.10001}{0.00001} \right) = 250 \cdot 10001 = \mathbf{2.500.250 \text{ ns} \approx 2,5 \text{ ms}}$$

2. Caso H massimo ($EAT = 200$ ns):

$$T > 200 \cdot \left(\frac{0.10001}{0.00001} \right) = 200 \cdot 10001 = \mathbf{2.000.200 \text{ ns} \approx 2,0 \text{ ms}}$$

Siccome la condizione richiede che la penalità superi una certa soglia, l'equazione definisce un limite inferiore. Pertanto, i valori trovati rappresentano un **valore minimo** per T.

C) Effetti della modifica della pagina e della struttura ($p1, p2$):

- **Modifica della dimensione della pagina: SÌ, varia il numero di PF.** Aumentare la dimensione della pagina inizialmente riduce i Page Fault grazie al principio di località spaziale. Tuttavia, se aumentata eccessivamente, aumenta la frammentazione interna e diminuisce il numero di frame fisici assegnabili al processo in RAM, innescando l'effetto opposto (aumento dei PF e potenziale thrashing).
- **Modifica di p1 e p2 (senza cambiare d): NO, il numero di PF non varia.** La dimensione d rappresenta l'offset, che definisce in modo univoco la dimensione effettiva della pagina fisica (2^d). I parametri $p1$ e $p2$ determinano esclusivamente come l'indirizzo logico viene sezionato per l'esplorazione gerarchica della struttura dati della Page Table nella memoria. Una variazione del bilanciamento tra $p1$ e $p2$ altera unicamente lo spazio in RAM richiesto dai metadati del sistema operativo, ma il raggruppamento degli indirizzi all'interno delle pagine logiche rimane strutturalmente immutato.

Domanda 2 (File System Unix-like, Inodes e Blocchi)

A) Valori minimo e massimo per la dimensione del blocco X:

La dimensione X deve essere una potenza del 2. Esistono due vincoli architetturali che definiscono il range di X .

1. Calcolo del limite inferiore (X_{min}): Vincolo di indirizzabilità

La partizione ha dimensione 800 GB $\approx 800 \times 2^{30}$ Byte. I puntatori sono di 32 bit, ovvero possono indirizzare un massimo di 2^{32} blocchi logici. Il numero totale di blocchi nel disco deve essere minore o uguale al massimo indirizzabile:

$$\frac{800 \times 2^{30}}{X} \leq 2^{32} \implies X \geq \frac{800 \times 2^{30}}{2^{32}} = \frac{800}{4} = 200 \text{ Byte}$$

Essendo X una potenza di 2, il valore minimo valido è $X_{min} = 256 \text{ Byte}$.

2. Calcolo del limite superiore (X_{max}): Vincolo di spazio per 100 file in tripla indirezione

La partizione deve ospitare 100 file che "entrano" nell'indicizzazione tripla. Un file innesca il livello triplo quando esaurisce i puntatori diretti (10), singoli ($X/4$) e doppi ($(X/4)^2$). Il numero di blocchi dati per "entrare" appena nell'indirezione tripla è:

$$\text{Blocchi Dato}_{th} = 10 + \frac{X}{4} + \left(\frac{X}{4}\right)^2 + 1$$

I blocchi indice necessari per supportare questa dimensione minima di innesco sono:

$$\text{Blocchi Indice}_{th} = 1(\text{singolo}) + 1(\text{doppio ext}) + \frac{X}{4}(\text{doppi int}) + 1(\text{triplo}) + 1(\text{doppio in triplo}) + 1(\text{singolo in triplo})$$

Invece di risolvere complessi polinomi, verifichiamo le classiche potenze di 2 sapendo che il totale per 100 file deve essere $\leq 800 \text{ GB}$:

- **Prova con $X = 4096 \text{ B (4 KB)}$:** Puntatori per blocco $N_c = 1024$.
 Blocchi Dato = $10 + 1024 + 1024^2 + 1 \approx 1.049.611$ blocchi.
 Spazio totale per 1 file $\approx 1.049.611 \times 4096 \text{ B} \approx 4,3 \text{ GB}$.
 Spazio per 100 file $\approx 430 \text{ GB}$. Poiché $430 \text{ GB} \leq 800 \text{ GB}$, la condizione è **Soddisfatta**.
- **Prova con $X = 8192 \text{ B (8 KB)}$:** Puntatori per blocco $N_c = 2048$.
 Blocchi Dato = $10 + 2048 + 2048^2 + 1 \approx 4.196.363$ blocchi.
 Spazio totale per 1 file $\approx 4.196.363 \times 8192 \text{ B} \approx 34,3 \text{ GB}$.
 Spazio per 100 file $\approx 3430 \text{ GB}$. Questo **supera i 800 GB** disponibili.

Pertanto, il valore massimo valido è $X_{\max} = 4096$ Byte (4 KB).

B) Calcolo dei blocchi dato e indice per file da 10381,25 KB:

Dimensione file in byte = $10381,25 \times 1024 = 10.630.400$ Byte.

Caso 1: Con $X_{\max} = 4096$ Byte (1024 puntatori/blocco)

- **Blocchi dato:** $\lceil 10.630.400/4096 \rceil = 2596$ blocchi dato.
- **Blocchi indice:** I 2596 blocchi sono così mappati:
 - 10 tramite indici diretti (restano 2586 blocchi).
 - 1024 tramite 1 blocco singolo indiretto (restano 1562 blocchi).
 - I 1562 restanti cadono nel doppio indiretto. Servono $\lceil 1562/1024 \rceil = 2$ blocchi interni di secondo livello, puntati da 1 blocco doppio esterno.

Totale blocchi indice = $1(\text{singolo}) + 1(\text{doppio ext}) + 2(\text{doppi int}) = 4$ blocchi indice.

Caso 2: Con $X_{\min} = 256$ Byte (64 puntatori/blocco)

- **Blocchi dato:** $\lceil 10.630.400/256 \rceil = 41.525$ blocchi dato.
- **Blocchi indice:** I 41525 blocchi sono così mappati:
 - 10 tramite indici diretti.
 - 64 tramite 1 blocco singolo indiretto.
 - $64 \times 64 = 4096$ coperti interamente dal doppio indiretto, che richiede: 1 blocco esterno e 64 blocchi interni (Totale 65).
 - I restanti $41.525 - (10 + 64 + 4096) = 37.355$ cadono nel triplo indiretto.
 - Per 37.355 blocchi in tripla indirezione servono: $\lceil 37.355/64 \rceil = 584$ blocchi singoli interni. A loro volta, questi 584 blocchi richiedono $\lceil 37.355/4096 \rceil = 10$ blocchi doppi interni. A cui si somma 1 blocco triplo esterno.

Totale blocchi indice = $1(\text{singolo}) + 65(\text{doppio}) + 1(\text{triplo ext}) + 10(\text{doppi int}) + 584(\text{singoli int}) = 661$ blocchi indice.

Domanda 3 (Link Dinamico, IPT e TLB)

A) Dynamic Loading e Dynamic Linking:

Occorre analizzare le due dipendenze separatamente:

- **È possibile caricare dinamicamente senza link dinamico? SÌ.**
Un programma può essere interamente collegato staticamente (static linking), ma il programmatore può strutturare il codice in modo tale che specifiche routine o overlay vengano caricati in memoria principale solo nel momento in cui vengono esplicitamente invocati a runtime (dynamic loading).
- **Il dynamic linking richiede il dynamic loading? SÌ.**
Il link dinamico funziona inserendo nel codice eseguibile un frammento detto *stub*. Quando la funzione di libreria viene invocata, lo stub verifica se la libreria è già presente in memoria. Qualora non lo fosse, il sistema operativo deve necessariamente **caricarla dinamicamente** (dynamic loading) in RAM per poterne eseguire il codice. Pertanto, la tecnica del link dinamico fa intrinsecamente affidamento sul supporto al caricamento dinamico.

B) Inverted Page Table (IPT) e differenze con Hashed PT:

- **Necessità dell'Hash nell'IPT:**

In un'IPT pura, la tabella contiene una riga per ogni frame fisico di RAM. Per tradurre una coppia (PID, Virtual Page Number), il sistema dovrebbe scansionare sequenzialmente l'intera tabella. Questo approccio ha una complessità temporale inaccettabile, pari a $O(N)$ accessi in memoria. L'aggiunta di una tabella di Hash permette di calcolare l'indice corrispondente in tempo (quasi) costante $O(1)$, abbattendo drasticamente il ritardo di traduzione.

- **Differenza tra IPT+Hash e Hash PT standard:**

La differenza fondamentale è **strutturale** e legata all'indirizzamento.

In un'IPT, la dimensione della tabella è vincolata esclusivamente alla quantità di *memoria fisica* installata, in quanto l'indice della tabella coincide letteralmente con il numero del frame fisico.

In una **Hash PT standard**, invece, si esegue l'hash sul numero di pagina virtuale. La dimensione della tabella di hash è indipendente dalla RAM fisica; ogni cella punta a una lista concatenata (per gestire le collisioni) in cui i nodi contengono esplicitamente il mapping verso il frame fisico. La Hash PT scala quindi in base alla dimensione dello spazio logico gestito, non di quello fisico.

C) CPU dotata di TLB e processi simultanei:

La TLB **può contenere** entry appartenenti a più processi simultaneamente.

Motivazione: Sebbene in architetture hardware datate o basilari la TLB fosse costretta a contenere i dati di un solo processo (obbligando a un *TLB flush* ad ogni context switch), le CPU moderne (inclusa l'architettura MIPS utilizzata in OS161) implementano in ogni entry della TLB un campo specifico denominato **ASID (Address Space Identifier)**. Questo identificatore hardware (analogo al PID) etichetta univocamente a quale processo appartiene una determinata traduzione logico-fisica. Durante l'accesso in memoria, la MMU verifica che l'indirizzo logico corrisponda e che l'ASID della TLB combaci con quello del processo attualmente in esecuzione, garantendo sicurezza ed evitando il flush.

Domanda 4 (Configurazione e Compilazione Kernel)

A) Sufficienza delle operazioni per la compilazione del file `syscall/exam.c`:

NO, le azioni non sono sufficienti.

Motivazione: L'inserimento di `defoption exam optfile exam syscall/exam.c` nel file `conf.kern` serve unicamente a "dichiarare" l'esistenza di una nuova opzione e a legarla al relativo file sorgente. Tuttavia, il file di configurazione **EXAM** (che è una semplice copia di DUMBVM) non contiene esplicitamente la direttiva di attivazione dell'opzione. Senza aggiungere la riga `options exam` all'interno del file **EXAM**, il programma `./config` manterrà l'opzione disabilitata e **non** aggiungerà `syscall/exam.c` al Makefile. Il comando `bmake` salterà la sua compilazione.

B) Quale file viene generato e quando:

Viene generato automaticamente il file header `opt-exam.h`.

Questo file viene generato **sempre** al lancio di `./config EXAM`, indipendentemente dal fatto che l'istruzione `options exam` sia presente o meno nel file di configurazione.

C) Contenuto del file generato:

Il file `opt-exam.h` contiene la definizione di una macro del preprocessore C associata all'opzione, che riflette il suo stato di attivazione:

- Se `options exam` non è presente nel file **EXAM**, il file conterrà: `#define OPT_EXAM 0`
- Se `options exam` è presente nel file **EXAM**, il file conterrà: `#define OPT_EXAM 1`

D) Inserimento di `exam_init()` e utilizzo delle macro:

Il codice proposto nella traccia utilizza la macro `#ifdef OPT_EXAM`. Questa istruzione è **logicamente errata** in OS161.

Motivazione: Poiché il file `opt-exam.h` viene generato in ogni caso (come discusso al punto B) e contiene sempre una `#define OPT_EXAM` (che valga 0 o che valga 1), la direttiva `#ifdef` (che controlla solo l'esistenza della macro, non il suo valore) risulterà **sempre vera**. Di conseguenza, la chiamata a `exam_init()` verrebbe compilata anche a opzione disabilitata (valore 0), causando un errore in fase di link (poiché il file sorgente non è stato compilato).

Per fare in modo che l'istruzione sia presa in considerazione solo se l'opzione è abilitata, occorre testare il valore logico della macro utilizzando la direttiva `#if`:

```
#if OPT_EXAM
    exam_init();
#endif
```

Inclusione necessaria:

Affinché il file `main.c` (o qualsiasi altro file sorgente) possa "vedere" il valore della macro `OPT_EXAM`, è assolutamente necessario aggiungere all'inizio del file la direttiva di inclusione dell'header autogenerato:

```
#include "opt-exam.h"
```

Domanda 5 (Processi, Trapframe e I/O)

A) Differenza tra `switchframe` e `trapframe`:

In OS161, queste due strutture hanno scopi e contesti di utilizzo completamente opposti:

- **Trapframe:** È la struttura dati (salvata sullo stack del kernel) che cattura l'intero stato della CPU (tutti i registri utente) nel momento in cui avviene una *Trap* (un passaggio da User Mode a Kernel Mode dovuto a interrupt, eccezioni o system call).
- **Switchframe:** È la struttura dati utilizzata per effettuare un *Context Switch* (cambio di contesto) tra due **Kernel Thread**. Salva unicamente lo stato dei registri "callee-save" (registri che il compilatore C si aspetta rimangano inalterati dopo una chiamata a funzione) e non ha nulla a che fare con lo spazio utente.

Risposte specifiche per i tre scenari:

1. Avviare un processo utente: **Trapframe** (Si costruisce un trapframe fittizio per ingannare la CPU ed eseguire un rientro in User Mode tramite l'istruzione `jr` o `eret`).
2. Gestire una chiamata di sistema: **Trapframe** (La syscall è una trap generata volontariamente dal processo utente).
3. Per la `thread_fork`: **Switchframe** (Inizializza lo stato del nuovo thread nel kernel affinché possa essere "ripreso" dallo scheduler).

B) Mappatura dei dispositivi di I/O (`kseg1` vs `kseg0`):

I dispositivi di I/O utilizzano la tecnica del *Memory-Mapped I/O* (MMIO), in cui i registri di controllo e dati della periferica sono mappati su specifici indirizzi di memoria fisica.

- Devono risiedere in **kseg1** perché questo segmento è **uncached** (non utilizza la memoria cache della CPU). Ogni operazione di load/store va direttamente sul bus hardware verso la periferica.

- **NON** possono essere mappati in **kseg0** perché **kseg0** è **cached**. Se un registro di I/O finisse in cache, una scrittura della CPU verrebbe trattenuta in cache (write-back) senza mai raggiungere la periferica, e una lettura restituirebbe il vecchio valore presente in cache invece del dato aggiornato generato in tempo reale dall'hardware.

C) Chiamata `write()` con buffer in `kseg0`:

NO, non è possibile (l'operazione verrà respinta).

Motivazione: Lo spazio di indirizzamento MIPS è diviso in **kuseg** (User Space, indirizzi inferiori a 0x80000000) e Kernel Space (che include **kseg0** e **kseg1**, da 0x80000000 in su). Un processo in User Mode non dispone dei privilegi hardware per accedere a **kseg0**. Quando la chiamata a sistema **write** viene invocata, il codice del kernel verificherà la validità del puntatore **buf** utente (generalmente tramite la funzione di sicurezza **copyin**). Rilevando che il puntatore tenta di indirizzare la memoria protetta del kernel, la system call fallirà restituendo un codice di errore di violazione di memoria (es. **EFAULT**), senza eseguire alcuna scrittura su disco.